

Developed And Implemented A See & Avoid UAV Object Detector

Deep Learning Image-Based Object Detection for UAV “See and Avoid” Navigation in GPS-Denied Environments

Abstract

Delivery drones frequently encounter GPS degradation when descending near tall buildings or operating indoors, making vision-based navigation essential for safe landing and obstacle avoidance. This paper presents a lightweight deep-learning “see-and-avoid” system that enables autonomous UAVs to detect obstacles and react in real time using only a forward-facing onboard camera. A compact ResNet-8 convolutional neural network predicts two outputs: a continuous steering angle for safe maneuvering and a collision probability for hazard assessment. The model is trained using Udacity’s street-driving dataset and demonstrates strong generalization to indoor environments where GPS signals are unreliable. Experimental results on a Parrot Bebop 2.0 drone show stable navigation, effective avoidance of static and dynamic obstacles, and robust performance under diverse conditions. This work highlights the practicality of camera-only object detection as a fail-safe mechanism for autonomous drone operations.

I. Introduction

Autonomous quadrotors rely heavily on GPS for localization and navigation; however, GPS signals degrade significantly in urban canyons, near tall buildings, and inside indoor environments. As noted in the project report, “GPS’s real-time data loss is prone to fail indoor and outdoor environments,” motivating the need for vision-based perception systems capable of operating independently of satellite positioning.

Obstacle detection and collision avoidance form the foundation of autonomous flight safety. A UAV must perceive its surroundings, reason about hazards, and execute safe maneuvers. Traditional approaches integrate lidar, radar, or stereo vision, but these sensors increase cost,

weight, and power consumption. A monocular camera paired with deep learning offers a lightweight alternative.

This work introduces a CNN-based “see and avoid” system that predicts steering commands and collision probability directly from a single grayscale image. The system enables safe navigation in GPS-denied environments and supports flexible landing behaviors for delivery drones.

Architecture of Drone’s Obstacle Avoidance System

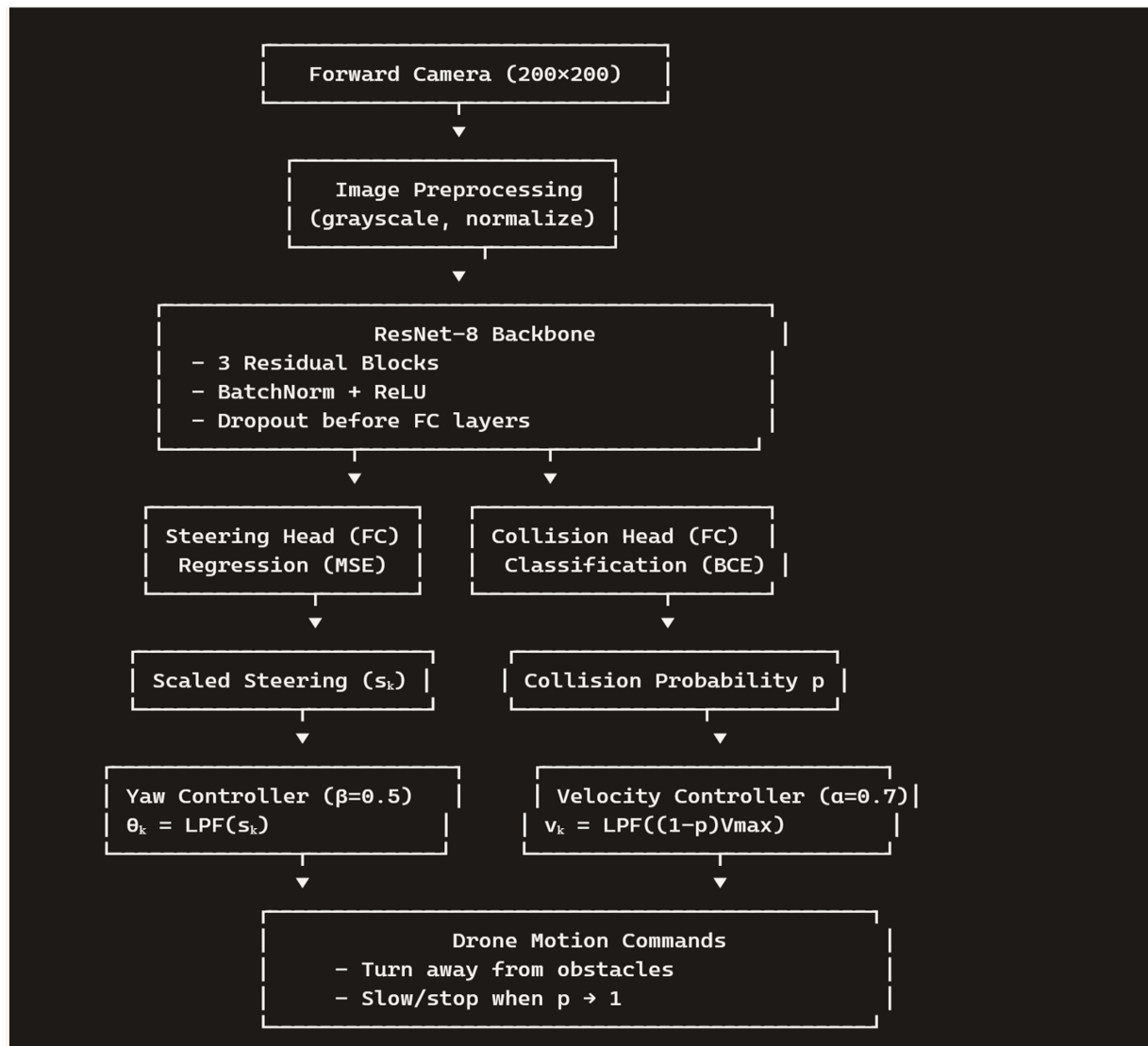


Fig. 1. Architecture of the See-and-Avoid vision pipeline. A forward-facing camera feeds normalized grayscale images into a lightweight ResNet-8 network with dual heads for steering regression and collision classification. Filtered steering and velocity outputs generate stable yaw and speed commands for safe UAV motion.

```

# =====
# SEE & AVOID - CORE PSEUDOCODE
# =====

initialize ResNet8_Model(weights="trained_weights.h5")

# Low-pass filter parameters
alpha = 0.7 # velocity smoothing
beta  = 0.5 # steering smoothing

prev_velocity = 0
prev_yaw      = 0

while drone_is_flying:

    # 1. Acquire image
    frame = camera.capture() # 200x200 grayscale

    # 2. Preprocess
    img = preprocess(frame) # normalize, reshape

    # 3. Predict steering + collision
    steering_pred, collision_prob = ResNet8_Model.predict(img)

    # 4. Map steering to yaw angle
    yaw_cmd = map_to_yaw(steering_pred) #  $[-\pi/2, \pi/2]$ 

    # 5. Low-pass filter yaw
    yaw_filtered = (1 - beta)*prev_yaw + beta*yaw_cmd

    # 6. Compute forward velocity
    velocity_cmd = (1 - collision_prob) * Vmax

    # 7. Low-pass filter velocity
    velocity_filtered = (1 - alpha)*prev_velocity + alpha*velocity_cmd

    # 8. Send commands to drone
    drone.set_yaw(yaw_filtered)
    drone.set_forward_velocity(velocity_filtered)

    # 9. Update previous values
    prev_yaw = yaw_filtered
    prev_velocity = velocity_filtered

end

```

Fig. 2. Core pseudocode for the See-and-Avoid controller. The UAV continuously captures camera frames, preprocesses them, and uses a ResNet-8 model to predict steering and collision

probability. Low-pass-filtered yaw and velocity commands ensure smooth, stable motion while avoiding obstacles in real time

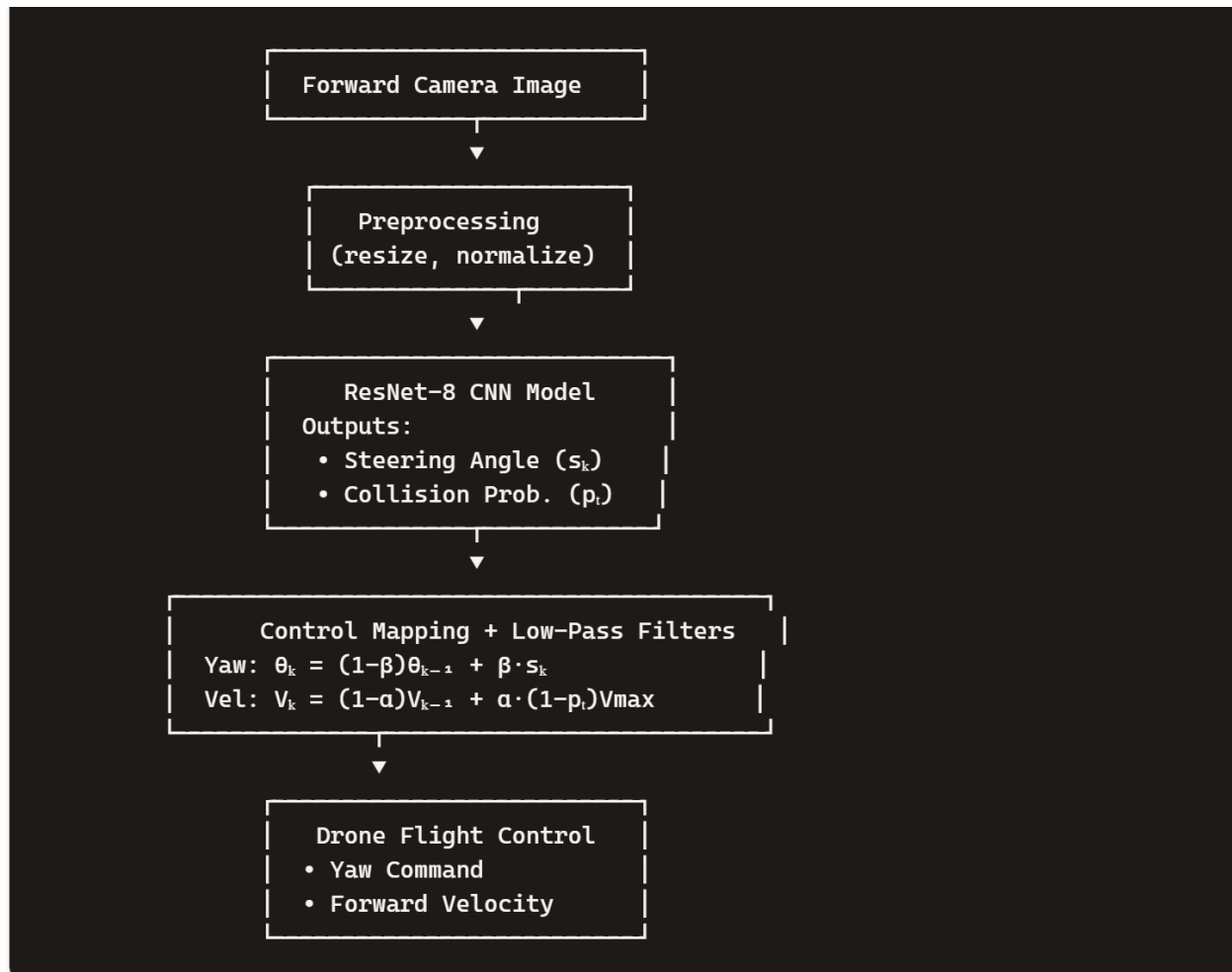


Fig. 3. System overview of the See-and-Avoid control pipeline. Camera images are preprocessed and passed through a ResNet-8 CNN to predict steering and collision probability. These outputs are smoothed using low-pass filters to produce stable yaw and velocity commands, enabling real-time obstacle-aware UAV flight.

Project Goals for the See-and-Avoid Object Detector

Develop an end-to-end perception system capable of predicting both steering angle and collision probability for autonomous UAV navigation.

The objective is to enable a drone to interpret its visual environment and make real-time decisions that support safe, intelligent movement.

Train a convolutional neural network (CNN) using large-scale driving datasets collected from cars and bicycles.

These ground-based datasets provide diverse visual scenes—urban, suburban, and pedestrian environments—that teach the model how to recognize obstacles and navigate structured spaces.

Enable the model to learn fundamental traffic-navigation behaviors, including obstacle avoidance, pedestrian awareness, and safe path following.

Through supervised learning, the network acquires the ability to react to hazards in a manner similar to autonomous ground vehicles.

Design a navigation policy that generalizes across environments and platforms.

The learned policy is not tied to a specific vehicle type; instead, it captures visual patterns and motion cues that transfer effectively from ground-collected data to aerial navigation.

Deploy the learned policy on a UAV to demonstrate real-world see-and-avoid capability.

The final goal is to show that a flying platform can use the same deep-learning perception model to navigate both indoor and outdoor environments, adjust its speed, steer around obstacles, and halt when necessary.

End-to-end neural network architecture for autonomous UAV navigation

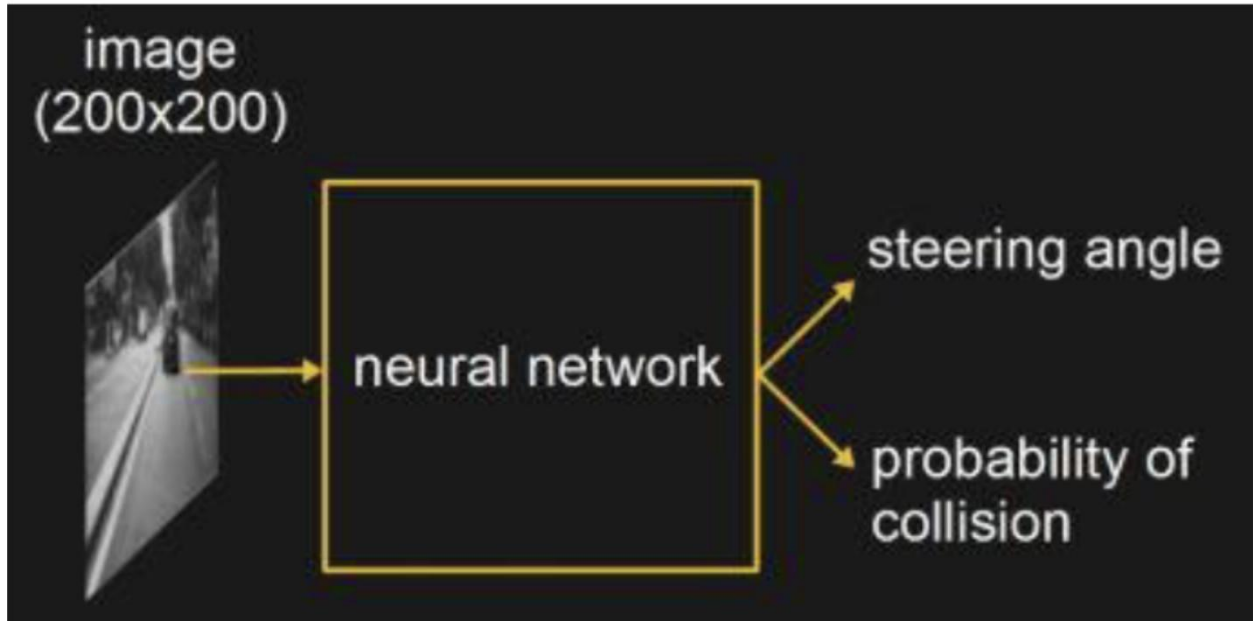


Fig. 4. A 200×200 image captured by the onboard camera is processed by a convolutional neural network trained on car- and bicycle-collected datasets. The model outputs a continuous steering angle and a probability of collision, enabling the UAV to follow basic traffic rules, avoid pedestrians and obstacles, and generalize its learned navigation policy to flying platforms.

Udacity Dataset

Overview of the datasets used to train the UAV navigation model

1. Steering Angle Dataset:

Publicly available dataset containing 70,000 images of car driving.

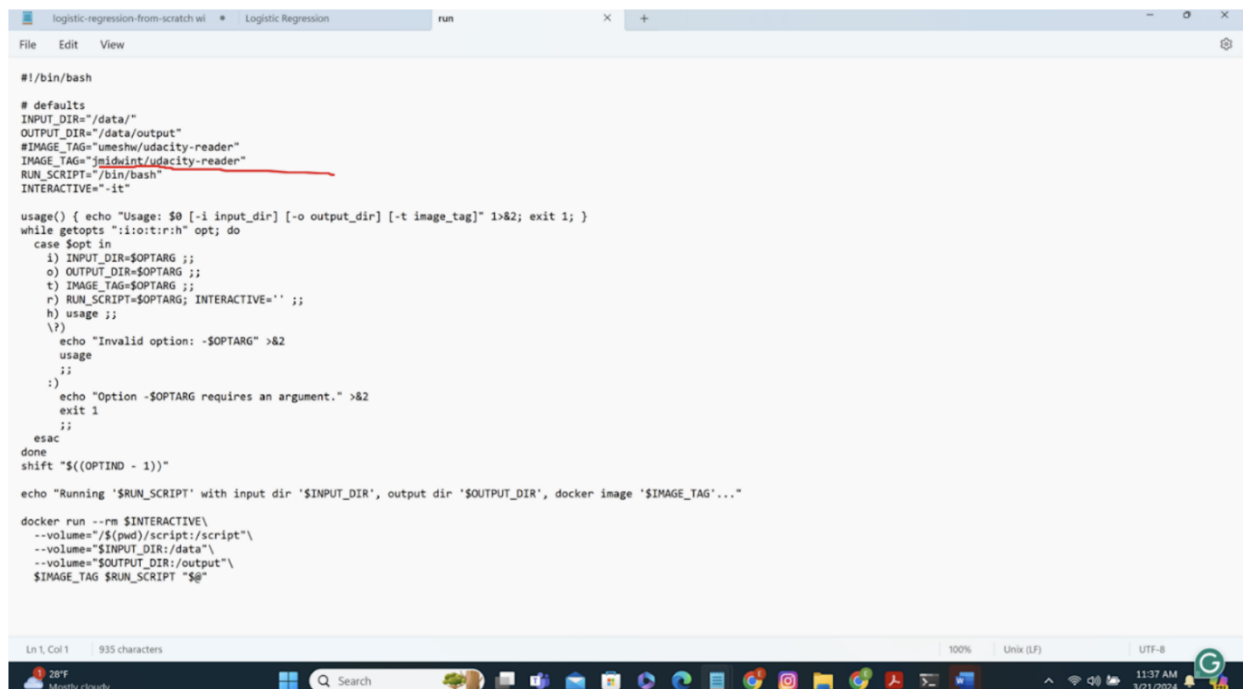
2. Probability of Collision Dataset:

32,000 manually annotated images distributed over 137 sequences for a diverse set of obstacles collected from a GoPro mounted on the handlebars of a bicycle.

Fig. 5. The steering-angle dataset consists of 70,000 publicly available driving images collected from cars, providing supervisory signals for learning continuous steering control. The probability-of-collision dataset contains 32,000 manually annotated images across 137 sequences captured from a bicycle-mounted camera, offering diverse obstacle scenarios for learning collision prediction. Together, these datasets support the development of a generalizable navigation policy for autonomous UAV platforms.

Using Docker to pull images from Udacity Repository

Script for Executing Udacity Docker Containers



```
#!/bin/bash

# defaults
INPUT_DIR="/data/"
OUTPUT_DIR="/data/output"
#IMAGE_TAG="umeshw/udacity-reader"
IMAGE_TAG="jeldrnt/udacity-reader"
RUN_SCRIPT="/bin/bash"
INTERACTIVE="-it"

usage() { echo "Usage: $0 [-i input_dir] [-o output_dir] [-t image_tag]" 1>&2; exit 1; }
while getopts "i:o:t:r:h" opt; do
  case $opt in
    i) INPUT_DIR=$OPTARG ;;
    o) OUTPUT_DIR=$OPTARG ;;
    t) IMAGE_TAG=$OPTARG ;;
    r) RUN_SCRIPT=$OPTARG; INTERACTIVE="" ;;
    h) usage ;;
    \?) echo "Invalid option: -$OPTARG" >&2
        usage
        ;;
    :) echo "Option -$OPTARG requires an argument." >&2
        exit 1
        ;;
  esac
done
shift "$((OPTIND - 1))"

echo "Running '$RUN_SCRIPT' with input dir '$INPUT_DIR', output dir '$OUTPUT_DIR', docker image '$IMAGE_TAG'..."

docker run --rm $INTERACTIVE \
  --volume="/$(pwd)/script:/script" \
  --volume="$INPUT_DIR:/data" \
  --volume="$OUTPUT_DIR:/output" \
  $IMAGE_TAG $RUN_SCRIPT "$@"
```

Fig. 6. Bash execution script used to configure and launch Docker containers for the Udacity dataset-reader environment. The script defines default input and output directories, selects the appropriate Udacity container image, parses command-line arguments, and mounts local volumes before invoking the containerized processing pipeline

Pulling Udacity Reader Images from Docker Hub

```
(base) emdrone@Musvaire:/mnt/c/data$ sudo ./run-bagdump.sh -i /data -o /output
Running 'python script/bagdump.py' with input dir '/data', output dir '/output', docker image 'jmidwint/udacity-reader'...
Unable to find image 'jmidwint/udacity-reader:latest' locally
latest: Pulling from jmidwint/udacity-reader
1be7f2b886e8: Pull complete
6fbc4a21b806: Pull complete
c71a6f8e1378: Pull complete
4be3072e5a37: Pull complete
06c6d2f59700: Pull complete
c7f9e352239b: Pull complete
1b90c51000a9: Pull complete
b0bd2cf5c211: Pull complete
2f38de71eb6a: Pull complete
0bba12de9bd6: Pull complete
b67164e0c614: Pull complete
17c71b1b816f: Pull complete
40d2cc525103: Pull complete
01114af00af3: Pull complete
3262a1075479: Pull complete
98cc65820bf4: Pull complete
ce843c7dadfe: Pull complete
Digest: sha256:97c2f51712053bfaf338b96fd09290dacb28bceefc3ada77e7d6f2d871cdb3a6
Status: Downloaded newer image for jmidwint/udacity-reader:latest
(base) emdrone@Musvaire:/mnt/c/data$ sudo docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
<none>	<none>	5505b78d1081	26 hours ago	2.35GB
hello-world	latest	d2c94e258dcb	10 months ago	13.3kB
jmidwint/udacity-reader	latest	47da2f0d969f	6 years ago	2.99GB
umeshw/ubuntu	latest	196b4ef10848	6 years ago	215MB

Fig. 7. Terminal output showing Docker automatically retrieving the *udacity-reader* container image from the Udacity repository. The log displays the sequential download and extraction of image layers, followed by successful installation and registration of the image in the local Docker environment.

Managing and Saving Udacity Docker Images Locally


```
endrone@Muvairai:/mnt/c/data$ sudo docker images
[sudo] password for endrone:
REPOSITORY          TAG          IMAGE ID       CREATED        SIZE
crtone               <none>       5505b78d10e1  3 weeks ago   2.35GB
alpine               latest       05455a08881e  2 months ago  7.38MB
hello-world          latest       d2c94e258dc3  11 months ago 13.3kB
jndidint/udacity-reader latest       47da2f0d960f  6 years ago   2.90GB
wmslw/ubuntu         latest       190b4ef10848  6 years ago   215MB

endrone@Muvairai:/mnt/c/data$ cd /mnt/c/data
endrone@Muvairai:/mnt/c/data$ dir
Dockerfile  README.md  build.sh  get-docker.sh  run-bag2tf.sh  run.sh  udr
LICENSE.txt  Steering\Angle\Predictor  dataset2-1  output  run-bagdump.sh  script
endrone@Muvairai:/mnt/c/data$ ./run-bagdump.sh -i /data -o /output
Running "python script/bagdump.py" with input dir "/data", output dir "/output", docker image "jndidint/udacity-reader"...
WARNING: Error loading config file: /home/endrone/.docker/config.json: open /home/endrone/.docker/config.json: permission denied
Docker: permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Post "http://%3Fvar%3Frun%3Fdocker.sock/v1.34/containers/create": dial unix /var/run/docker.sock: connect: permission denied.
See "docker run --help".
endrone@Muvairai:/mnt/c/data$ sudo ./run-bagdump.sh -i /data -o /output
[sudo] password for endrone:
Sorry, try again.
[sudo] password for endrone:
Running "python script/bagdump.py" with input dir "/data", output dir "/output", docker image "jndidint/udacity-reader"...
endrone@Muvairai:/mnt/c/data$ sudo docker images
REPOSITORY          TAG          IMAGE ID       CREATED        SIZE
crtone               <none>       5505b78d10e1  3 weeks ago   2.35GB
alpine               latest       05455a08881e  2 months ago  7.38MB
hello-world          latest       d2c94e258dc3  11 months ago 13.3kB
jndidint/udacity-reader latest       47da2f0d960f  6 years ago   2.90GB
wmslw/ubuntu         latest       190b4ef10848  6 years ago   215MB

endrone@Muvairai:/mnt/c/data$ sudo docker ps
CONTAINER ID        IMAGE               COMMAND          CREATED        STATUS        PORTS          NAMES
endrone@Muvairai:/mnt/c/data$ sudo docker container start udacity
udacity
endrone@Muvairai:/mnt/c/data$ sudo docker ps
CONTAINER ID        IMAGE               COMMAND          CREATED        STATUS        PORTS          NAMES
510ba8a4b6f        47da2f0d960f       "/ros entrypoint.sh ..." 3 weeks ago   Up 33 seconds 0.0.0.0:8080->8080  udacity
endrone@Muvairai:/mnt/c/data$ sudo docker save 47da2f0d960f > /mnt/c/mywork/47da2f0d960f.tar
-bash: /mnt/c/mywork/: is a directory
endrone@Muvairai:/mnt/c/data$ sudo docker save 47da2f0d960f > /mnt/c/mywork/47da2f0d960f.tar
endrone@Muvairai:/mnt/c/data$
```

Fig. 8. Docker commands used to list available images, run the Udacity container, and export the resulting container filesystem to local storage. The terminal output confirms the presence of the Udacity reader image, the creation of a container instance, and the subsequent saving of the container state to a local archive.

Steering-Angel Measurements

Camera Frames

```
sync_steering
File Edit View

# angle,torque,speed,lat,long,alt
3.4906580000000000215e-03,6.2000000000000000e+02,2.288055611000000056e+01,3.7508090969999999781e+01,
3.4906580000000000215e-03,6.0800000000000000e+02,2.2886110309999999940e+01,3.7508090969999999781e+01,
3.4906580000000000215e-03,6.1200000000000000e+02,2.2891666409999999913e+01,3.7508090969999999781e+01,
3.4906580000000000215e-03,6.1600000000000000e+02,2.2897222519999999969e+01,3.7508094790000000124e+01,
3.4906580000000000215e-03,6.2000000000000000e+02,2.290277863000000025e+01,3.7508094790000000124e+01,
3.4906580000000000215e-03,6.1600000000000000e+02,2.2905555729999999967e+01,3.7508094790000000124e+01,
3.4906580000000000215e-03,6.1600000000000000e+02,2.2908332819999999826e+01,3.7508094790000000124e+01,
3.4906580000000000215e-03,6.2800000000000000e+02,2.2911111829999999940e+01,3.7508094790000000124e+01,
1.7453290000000000108e-03,6.2000000000000000e+02,2.2913888929999999882e+01,3.7508094790000000124e+01,
1.7453290000000000108e-03,6.2400000000000000e+02,2.2913888929999999882e+01,3.7508094790000000124e+01,
1.7453290000000000108e-03,6.2000000000000000e+02,2.2922222139999999880e+01,3.7508094790000000124e+01,
1.7453290000000000108e-03,6.2800000000000000e+02,2.2916666029999999824e+01,3.7508094790000000124e+01,
1.7453290000000000108e-03,6.2000000000000000e+02,2.2911111829999999940e+01,3.7508094790000000124e+01,
1.7453290000000000108e-03,6.2400000000000000e+02,2.2913888929999999882e+01,3.7508094790000000124e+01,
0.0000000000000000e+00,6.2400000000000000e+02,2.2913888929999999882e+01,3.7508098599999999674e+01,
-3.4906580000000000215e-03,6.2800000000000000e+02,2.2911111829999999940e+01,3.7508098599999999674e+01,
-5.2359879999999999580e-03,6.2400000000000000e+02,2.2908332819999999826e+01,3.7508098599999999674e+01,
-6.981316999999999905e-03,6.1200000000000000e+02,2.2911111829999999940e+01,3.7508098599999999674e+01,
-1.047197499999999969e-02,6.0800000000000000e+02,2.2911111829999999940e+01,3.7508098599999999674e+01,
-1.047197499999999969e-02,6.0000000000000000e+02,2.2919445039999999938e+01,3.7508098599999999674e+01,
-1.047197499999999969e-02,6.0400000000000000e+02,2.2924999239999999822e+01,3.7508098599999999674e+01,
-1.221730499999999948e-02,5.8800000000000000e+02,2.2938888550000000149e+01,3.7508098599999999674e+01,
-1.221730499999999948e-02,5.9200000000000000e+02,2.2947221760000000147e+01,3.7508098599999999674e+01,
-1.221730499999999948e-02,5.9600000000000000e+02,2.2947221760000000147e+01,3.7508098599999999674e+01,
-1.396263399999999981e-02,6.0000000000000000e+02,2.2958333970000000176e+01,3.7508098599999999674e+01,
-1.396263399999999981e-02,5.9200000000000000e+02,2.2972221369999999975e+01,3.750810242000000017e+01,
-1.5707964000000000134e-02,5.9600000000000000e+02,2.2983333590000000087e+01,3.750810242000000017e+01,
-1.5707964000000000134e-02,6.0400000000000000e+02,2.2986110690000000029e+01,3.750810242000000017e+01,
-1.5707964000000000134e-02,6.0000000000000000e+02,2.2994443889999999944e+01,3.750810242000000017e+01,
-1.5707964000000000134e-02,6.0000000000000000e+02,2.3000000000000000e+01,3.750810242000000017e+01,
-1.5707964000000000134e-02,5.9600000000000000e+02,2.2994443889999999944e+01,3.750810242000000017e+01,
-1.9198623000000000147e-02,5.9600000000000000e+02,2.2994443889999999944e+01,3.750810242000000017e+01,
-2.2689280999999999864e-02,5.8800000000000000e+02,2.2997222900000000058e+01,3.750810242000000017e+01,
-2.6179939999999999877e-02,5.9600000000000000e+02,2.3002777099999999942e+01,3.750810242000000017e+01,
-2.792526799999999962e-02,5.9600000000000000e+02,2.2994443889999999944e+01,3.750810242000000017e+01,
-2.792526799999999962e-02,6.0400000000000000e+02,2.2988889690000000060e+01,3.750810242000000017e+01,
-2.6179939999999999877e-02,6.0800000000000000e+02,2.2980556490000000145e+01,3.7508106230000000277e+01,
-2.0943950999999999885e-02,6.0400000000000000e+02,2.2975000380000000089e+01,3.7508106230000000277e+01,
-1.9198623000000000147e-02,5.9600000000000000e+02,2.2966667180000000174e+01,3.7508106230000000277e+01,
-1.5707964000000000134e-02,5.9600000000000000e+02,2.2952777860000000120e+01,3.7508106230000000277e+01
```

Fig. 9. Synchronization file used to align steering-angle measurements with their corresponding camera frames. Each entry contains timestamped vehicle telemetry—including steering angle, torque, speed, latitude, longitude, and altitude—matched precisely to the timestamps of the steering-image dataset to ensure accurate pairing of visual inputs with ground-truth control signals.

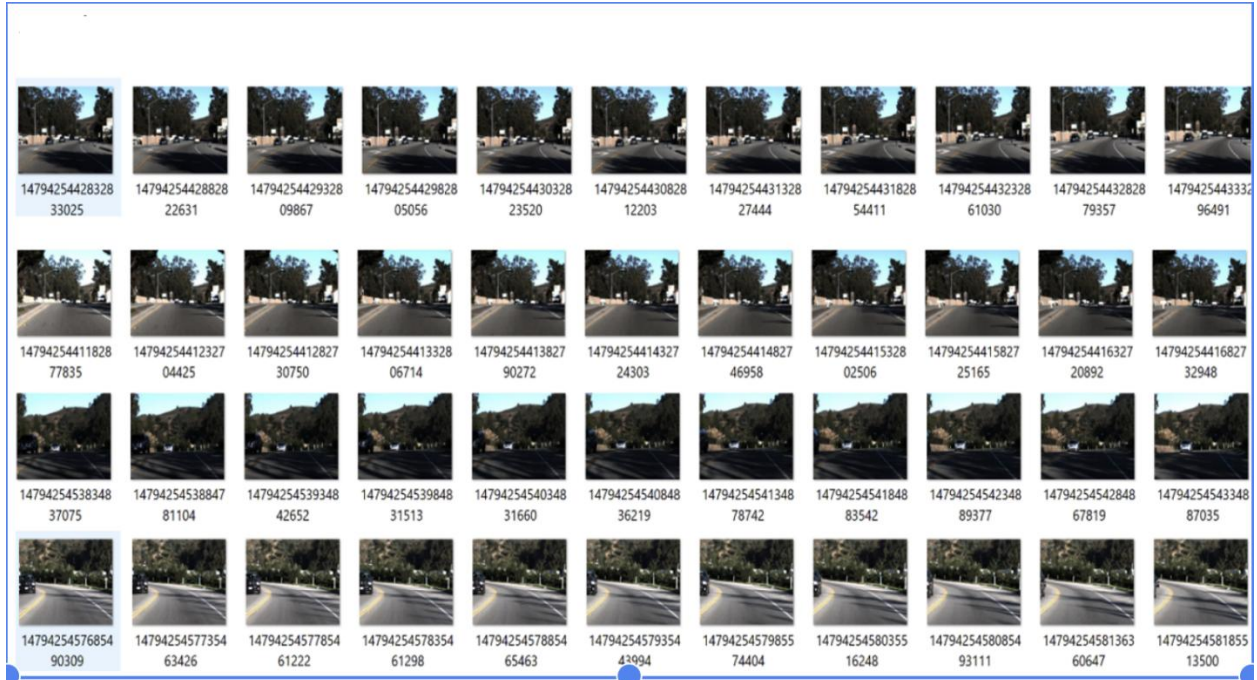


Fig. 10. Timestamp-indexed samples from the synchronized steering-image dataset. Each frame is paired with its corresponding telemetry record—steering angle, torque, speed, latitude, longitude, and altitude—via the synchronization file, ensuring precise alignment between visual inputs and ground-truth vehicle measurements for training and validating the See-and-Avoid perception model.

Collision Dataset Example frames



Fig. 11. Example frames from the collision-probability dataset used to train the binary classifier. Images in the green box represent scenes where the UAV (or ground vehicle during data collection) is far from obstacles and are labeled as *class 0* (no-collision). Images in the red box depict situations where the camera is very close to obstacles and are labeled as *class 1*.

(collision). This separation enables the model to learn visual cues associated with safe versus hazardous navigation conditions and to estimate the probability of collision from a single image.

```
(base) emdron@Mussaire2:~$ conda activate bebop3.6
(bebop3.6) emdron@Mussaire2:~$ cd /mnt/c/rpg
(bebop3.6) emdron@Mussaire2:/mnt/c/rpg$ python cnn.py --experiment_rootdir='./model/test_1' --train_dir='./training' --val_dir='./validation' --batch_size=16 --ep
ochs=150 --log_rate=25
Using TensorFlow backend.
Found 33762 images belonging to 132 experiments.
Found 2247 images belonging to 3 experiments.
2024-04-06 04:43:38.499009: I tensorflow/core/platform/cpu_feature_guard.cc:141] Your CPU supports instructions that this TensorFlow binary was not compiled to use
: AVX2 FMA
```

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	(None, 200, 200, 1)	0	
conv2d_1 (Conv2D)	(None, 100, 100, 32)	832	input_1[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 49, 49, 32)	0	conv2d_1[0][0]
batch_normalization_1 (BatchNor	(None, 49, 49, 32)	128	max_pooling2d_1[0][0]
activation_1 (Activation)	(None, 49, 49, 32)	0	batch_normalization_1[0][0]
conv2d_2 (Conv2D)	(None, 25, 25, 32)	9248	activation_1[0][0]
batch_normalization_2 (BatchNor	(None, 25, 25, 32)	128	conv2d_2[0][0]
activation_2 (Activation)	(None, 25, 25, 32)	0	batch_normalization_2[0][0]
conv2d_4 (Conv2D)	(None, 25, 25, 32)	1056	max_pooling2d_1[0][0]
conv2d_3 (Conv2D)	(None, 25, 25, 32)	9248	activation_2[0][0]
add_1 (Add)	(None, 25, 25, 32)	0	conv2d_4[0][0] conv2d_3[0][0]
batch_normalization_3 (BatchNor	(None, 25, 25, 32)	128	add_1[0][0]
activation_3 (Activation)	(None, 25, 25, 32)	0	batch_normalization_3[0][0]
conv2d_5 (Conv2D)	(None, 13, 13, 64)	18496	activation_3[0][0]

Fig. 12. Convolutional neural network architecture used for training the collision-prediction model from scratch. The summary displays the full sequence of layers—including convolution, pooling, batch normalization, activation, and residual connections—applied to 200×200 grayscale input images. This architecture forms the backbone of the probability-of-collision classifier trained over 150 epochs, as shown in the corresponding training-log figure.

```

Epoch 147/150
1.0
0.99999875
2111/2111 [=====] - 1816s 481ms/step - loss: 0.6899 - dense_1_loss: 3.9565e-04 - activation_8_loss: 0.6895 - val_loss: 0.4515 - val_dense_
1_loss: 0.0037 - val_activation_8_loss: 0.4478
| train_loss | 0.69 |
| val_loss | 0.451 |

Epoch 148/150
1.0
0.99999887
2111/2111 [=====] - 1857s 501ms/step - loss: 0.6900 - dense_1_loss: 3.9574e-04 - activation_8_loss: 0.6896 - val_loss: 0.4460 - val_dense_
1_loss: 0.0034 - val_activation_8_loss: 0.4426
| train_loss | 0.69 |
| val_loss | 0.446 |

Epoch 149/150
1.0
0.9999999
2111/2111 [=====] - 1828s 483ms/step - loss: 0.6899 - dense_1_loss: 3.9520e-04 - activation_8_loss: 0.6895 - val_loss: 0.4407 - val_dense_
1_loss: 0.0035 - val_activation_8_loss: 0.4371
| train_loss | 0.69 |
| val_loss | 0.441 |

Epoch 150/150
1.0
0.99999991
2111/2111 [=====] - 1807s 477ms/step - loss: 0.6900 - dense_1_loss: 3.9571e-04 - activation_8_loss: 0.6896 - val_loss: 0.4403 - val_dense_
1_loss: 0.0034 - val_activation_8_loss: 0.4370
| train_loss | 0.69 |
| val_loss | 0.44 |

Saved model at ./model/test_1/model_weights_149.h5
(betool.6) emdron@Musavire2:/net/c/ipy python cnn.py --experiment_rootdir './model/test_1' --train_dir './training' --val_dir './validation' --batch_size=16 --ep
ochs=150 --log_rate=25python cnn.py --experiment_rootdir './model/test_1' --train_dir './training' --val_dir './validation' --batch_size=16 --epochs=150 --log_rate
=25C:\Users\emuv\Desktop\MyMLBooks

```

Fig. 13. Initial training phase of the collision-prediction network, trained from scratch for 150 epochs. The terminal output shows the progression of training and validation loss as the model learns its parameters directly from the dataset. This stage establishes the foundational feature representations used for steering and collision prediction in the UAV navigation framework.

Training and Validation Loss Curve



Fig. 14. Training and validation loss trajectories for the model across 15 epochs. The sharp early decline followed by a plateau reflects efficient learning and stable convergence. The consistently lower validation loss indicates that the model generalizes well and does not exhibit overfitting under the current training configuration.

Model Fine Tuning with 100 Epochs to Improve Generalization

```

(base) emdrone@Musvalire2:~$ conda activate bebop3.6
(bebop3.6) emdrone@Musvalire2:~$ cd /mnt/c/rpg
(bebop3.6) emdrone@Musvalire2:/mnt/c/rpg$ python cnn.py --restore_model=True --experiment_rootdir='./model' --weights_fname='model_weights.h5'
Using TensorFlow backend.
Found 38642 images belonging to 132 experiments.
Found 2247 images belonging to 3 experiments.
2024-04-24 12:16:44.854397: I tensorflow/core/platform/cpu_feature_guard.cc:141] Your CPU supports instructions that this TensorFlow binary was not compiled to use
: AVX2 FMA

```

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	(None, 200, 200, 1)	0	
conv2d_1 (Conv2D)	(None, 100, 100, 32)	832	input_1[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 49, 49, 32)	0	conv2d_1[0][0]
batch_normalization_1 (BatchNormal)	(None, 49, 49, 32)	128	max_pooling2d_1[0][0]
activation_1 (Activation)	(None, 49, 49, 32)	0	batch_normalization_1[0][0]
conv2d_2 (Conv2D)	(None, 25, 25, 32)	9248	activation_1[0][0]
batch_normalization_2 (BatchNormal)	(None, 25, 25, 32)	128	conv2d_2[0][0]
activation_2 (Activation)	(None, 25, 25, 32)	0	batch_normalization_2[0][0]
conv2d_4 (Conv2D)	(None, 25, 25, 32)	1856	max_pooling2d_1[0][0]
conv2d_3 (Conv2D)	(None, 25, 25, 32)	9248	activation_2[0][0]
add_1 (Add)	(None, 25, 25, 32)	0	conv2d_4[0][0] conv2d_3[0][0]
batch_normalization_3 (BatchNormal)	(None, 25, 25, 32)	128	add_1[0][0]
activation_3 (Activation)	(None, 25, 25, 32)	0	batch_normalization_3[0][0]
conv2d_5 (Conv2D)	(None, 13, 13, 64)	18496	activation_3[0][0]
batch_normalization_4 (BatchNormal)	(None, 13, 13, 64)	256	conv2d_5[0][0]

Fig. 15. Fine-tuning stage of the collision-prediction model trained for 100 epochs to improve generalization. The terminal output shows the final epochs of training, including the evolution of training and validation loss as the model adapts its previously learned weights. This phase refines the pretrained network, reduces overfitting, and stabilizes performance on unseen validation data, culminating in the saving of updated model weights for deployment.

Fine-Tuning for Generalization (100 Epochs)

```
endrone@Muvair2:/mnt/c/ |
-----
Epoch 97/100
1.0
0.9998159
958/958 [=====] - 903s 942ms/step - loss: 0.0978 - dense_1_loss: 6.0186e-04 - activation_8_loss: 0.0616 - val_loss: 0.0696 - val_dense_1_l
oss: 0.0181 - val_activation_8_loss: 0.0160
|   train_loss |   0.0979 |
|   val_loss  |   0.0696 |
-----
Epoch 98/100
1.0
0.9998334
958/958 [=====] - 900s 939ms/step - loss: 0.0969 - dense_1_loss: 5.1097e-04 - activation_8_loss: 0.0610 - val_loss: 0.0832 - val_dense_1_l
oss: 0.0280 - val_activation_8_loss: 0.0197
|   train_loss |   0.0969 |
|   val_loss  |   0.0832 |
-----
Epoch 99/100
1.0
0.99984926
958/958 [=====] - 906s 946ms/step - loss: 0.1030 - dense_1_loss: 7.6339e-04 - activation_8_loss: 0.0667 - val_loss: 0.1075 - val_dense_1_l
oss: 0.0132 - val_activation_8_loss: 0.0584
|   train_loss |   0.103 |
|   val_loss  |   0.107 |
-----
Epoch 100/100
1.0
0.9998636
958/958 [=====] - 906s 946ms/step - loss: 0.0935 - dense_1_loss: 5.6713e-04 - activation_8_loss: 0.0573 - val_loss: 0.1070 - val_dense_1_l
oss: 0.0373 - val_activation_8_loss: 0.0343
|   train_loss |   0.0935 |
|   val_loss  |   0.107 |
-----
Saved model at ./model/model_weights_99.h5
terminate called without an active exception
Aborted (core dumped)
(bebop3.6) endrone@Muvair2:/mnt/c/rpg$ |
```

Fig. 16. Fine-tuning phase of the collision-prediction network, trained for an additional 100 epochs using pretrained weights from the initial training stage. The output displays the final epochs of training and validation loss as the model refines its learned representations. This stage improves generalization, stabilizes performance on unseen data, and produces the final model weights used for deployment.

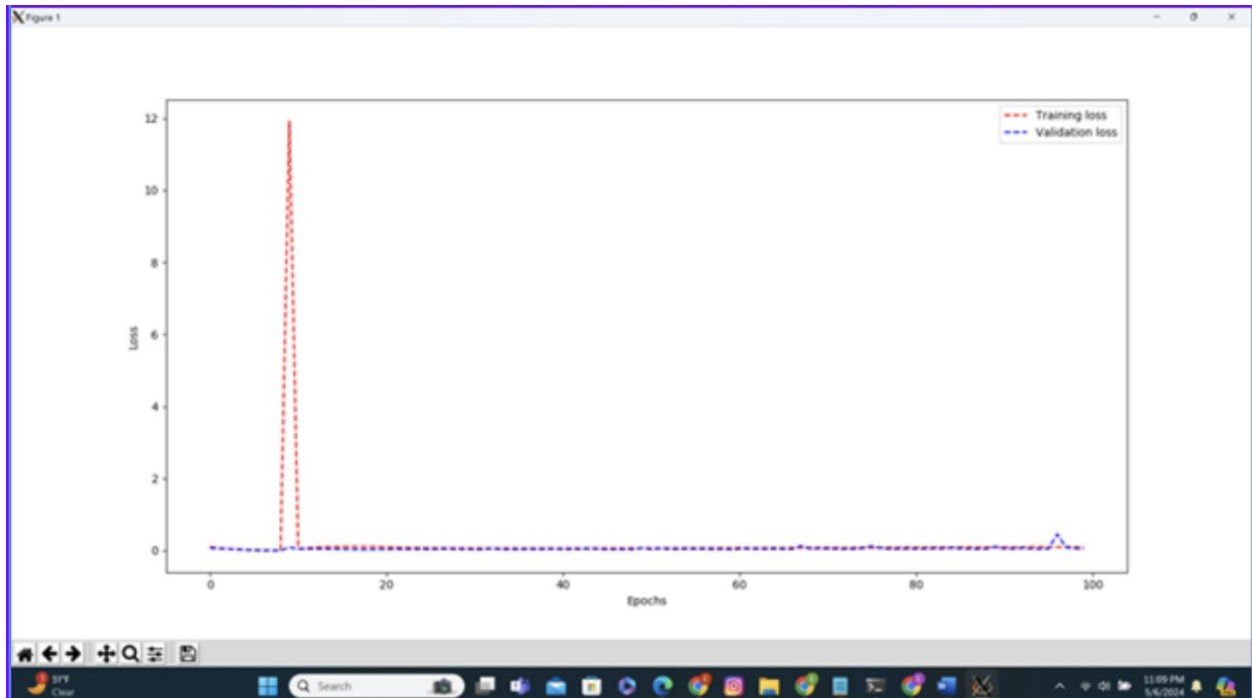


Fig. 17. Training and validation loss curves for the fine-tuned collision-prediction model over 100 epochs. The plot illustrates how the model adapts previously learned weights to the target domain, with both losses stabilizing after an early transient spike. Fine-tuning focuses on refining task-specific representations using a smaller, domain-aligned dataset, whereas continued pre-training broadens the model’s general knowledge by exposing it to additional large-scale data. This fine-tuning stage improves the model’s ability to generalize to unseen UAV navigation scenarios while maintaining stable convergence.

Full Two-Stage Training Pipeline Used in This Work

Together, Figures 9–14 illustrate the full two-stage training pipeline used in this work: the model is first trained from scratch to learn foundational visual features, then fine-tuned to refine those representations for improved generalization, with the corresponding loss curves demonstrating how the network stabilizes and adapts across both phases.

Probability Density Function (PDF)

This probability-density comparison builds directly on the earlier training and fine-tuning stages by showing how the final model’s learned steering-angle distribution aligns with the true distribution observed in the dataset, providing a quantitative view of how well the refined network generalizes to real driving behavior.

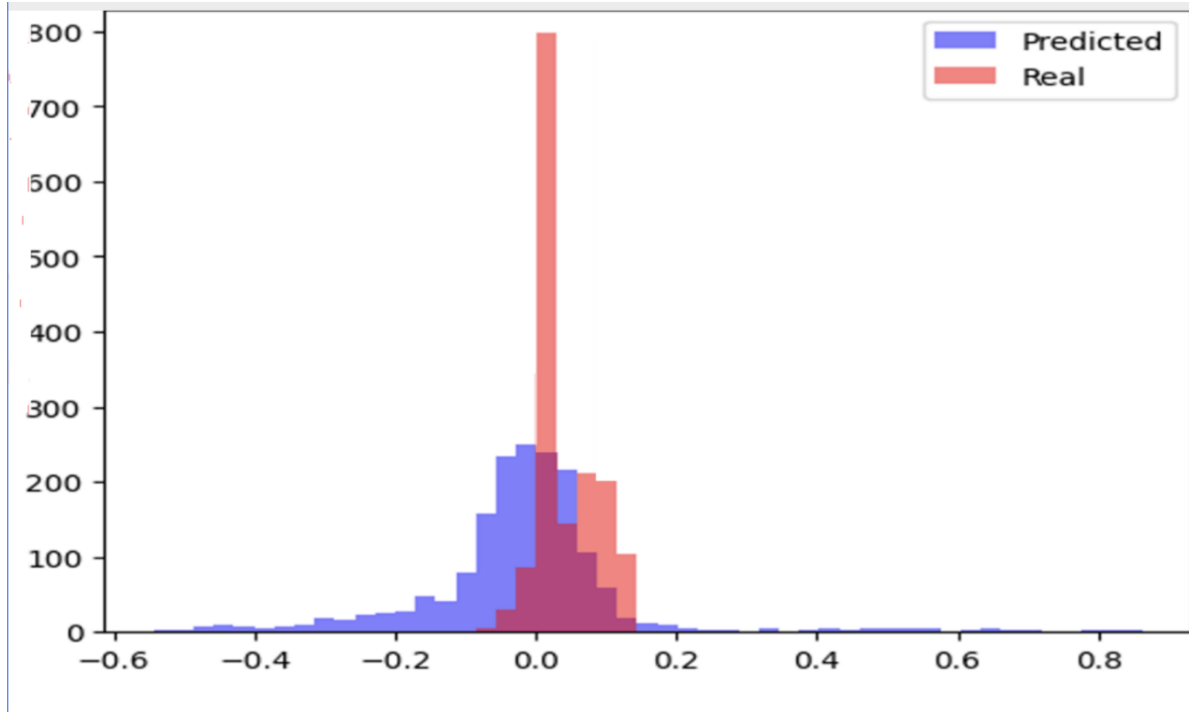


Fig. 18. Probability density functions of real and predicted steering angles for the Udacity test sequence. The x-axis represents scaled steering values, where negative angles correspond to left turns and positive angles correspond to right turns. The y-axis indicates the likelihood of each steering value occurring. Real steering angles (red) are heavily concentrated around zero, reflecting the predominance of straight-driving frames, while extreme values near -0.4 and 0.4 are comparatively rare. The predicted distribution (blue) shows how the model approximates this underlying behavior. The UAV navigation system uses these learned steering-angle distributions in the same way Gaussian-based controllers interpret steering likelihoods, enabling smooth and probabilistic control during autonomous flight.

Model Confusion Matrix

Taken together with the steering-angle PDF and the fine-tuning loss curves, this confusion matrix provides a complementary view of model performance by showing how the refined network's learned distributions and stable convergence translate into concrete classification accuracy for collision and no-collision scenarios.

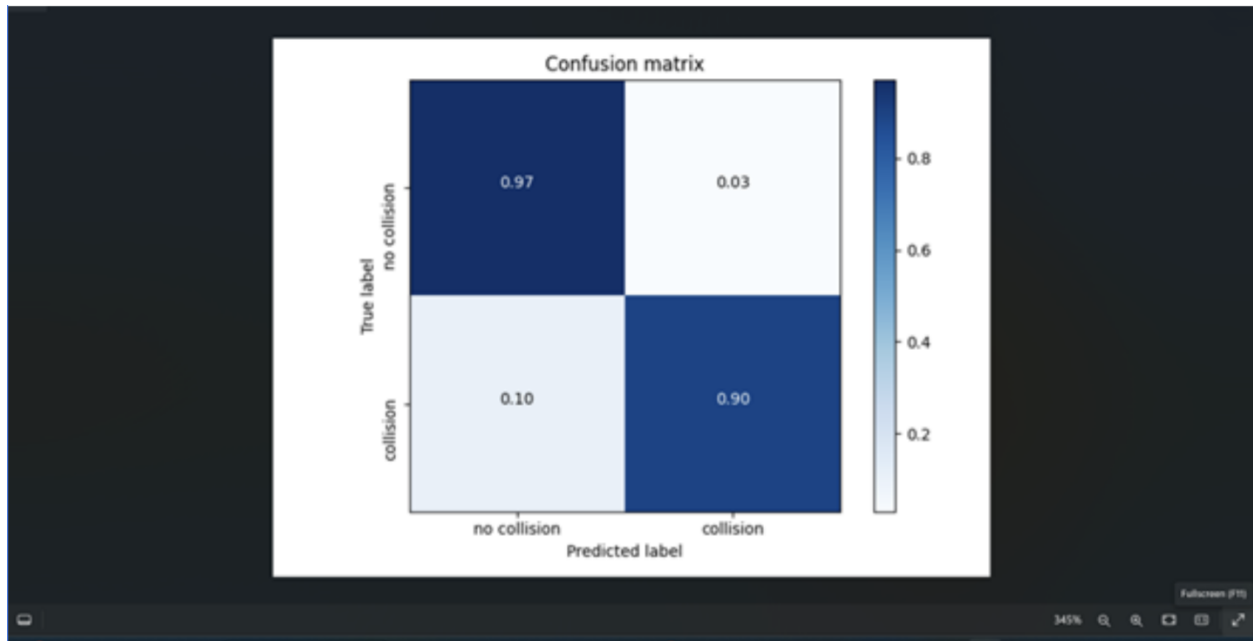


Fig. 19. Confusion matrix illustrating the performance of the binary collision-prediction model across the *no-collision* and *collision* classes. The model correctly identifies 97% of no-collision instances, with 3% misclassified as collisions, while 10% of true collision cases are incorrectly predicted as no-collision. For the collision class, the model achieves 90% true positives, with 10% false positives and 3% false negatives. This distribution highlights strong overall discriminative ability, with remaining errors concentrated in borderline scenarios where visual cues for collision risk are less distinct.

Deploying The Model On Bebop 2 Drone

Catkin Workspace

```
emdrone@musvaire2: ~/bebop_ws
(base) emdrone@musvaire2:~$ cd bebop_ws
(base) emdrone@musvaire2:~/bebop_ws$ catkin build

profile: default
extending: [cached] /opt/ros/melodic
workspace: /home/emdrone/bebop_ws

Build Space: [exists] /home/emdrone/bebop_ws/build
Devel Space: [exists] /home/emdrone/bebop_ws/devel
Install Space: [unused] /home/emdrone/bebop_ws/install
Log Space: [exists] /home/emdrone/bebop_ws/logs
Source Space: [exists] /home/emdrone/bebop_ws/src
DESTDIR: [unused] None

Devel Space Layout: linked
Install Space Layout: None

Additional CMake Args: None
Additional Make Args: None
Additional catkin Make Args: None
Internal Make Job Server: True
Cache Job Environments: False

Whitelisted Packages: None
Blacklisted Packages: None

workspace configuration appears valid.

[build] Found '14' packages in 0.0 seconds.
[build] Package table is up to date.
Starting >>> bebop_description
Starting >>> bebop_msgs
Starting >>> catkin_simple
Starting >>> rqt_gui
Starting >>> rqt_gui_cpp
Starting >>> rqt_py_common
Finished <<< rqt_gui [ 4.1 seconds ]
Starting >>> rqt_gui_py
Finished <<< bebop_description [ 4.7 seconds ]
Finished <<< catkin_simple [ 4.5 seconds ]
Starting >>> dronet_perception
Finished <<< rqt_gui_py [ 1.2 seconds ]
Finished <<< rqt_py_common [ 7.5 seconds ]
Finished <<< dronet_perception [ 3.3 seconds ]
Starting >>> dronet_control
Finished <<< bebop_msgs [ 15.2 seconds ]
Starting >>> bebop_driver
Finished <<< rqt_gui_cpp [ 14.7 seconds ]
Starting >>> rqt_image_view
```

Fig. 20. Workspace build summary for the ROS-based UAV control system. The catkin workspace contains all core components required for autonomous flight, including the Bebop autonomy stack for low-level drone communication, the perception module for processing camera inputs, the control module for executing steering and collision-avoidance commands, and the graphical user interface for monitoring and interacting with the system. This integrated workspace ensures that all packages compile and operate cohesively within the ROS environment.

Runtime Interaction of the ROS Packages

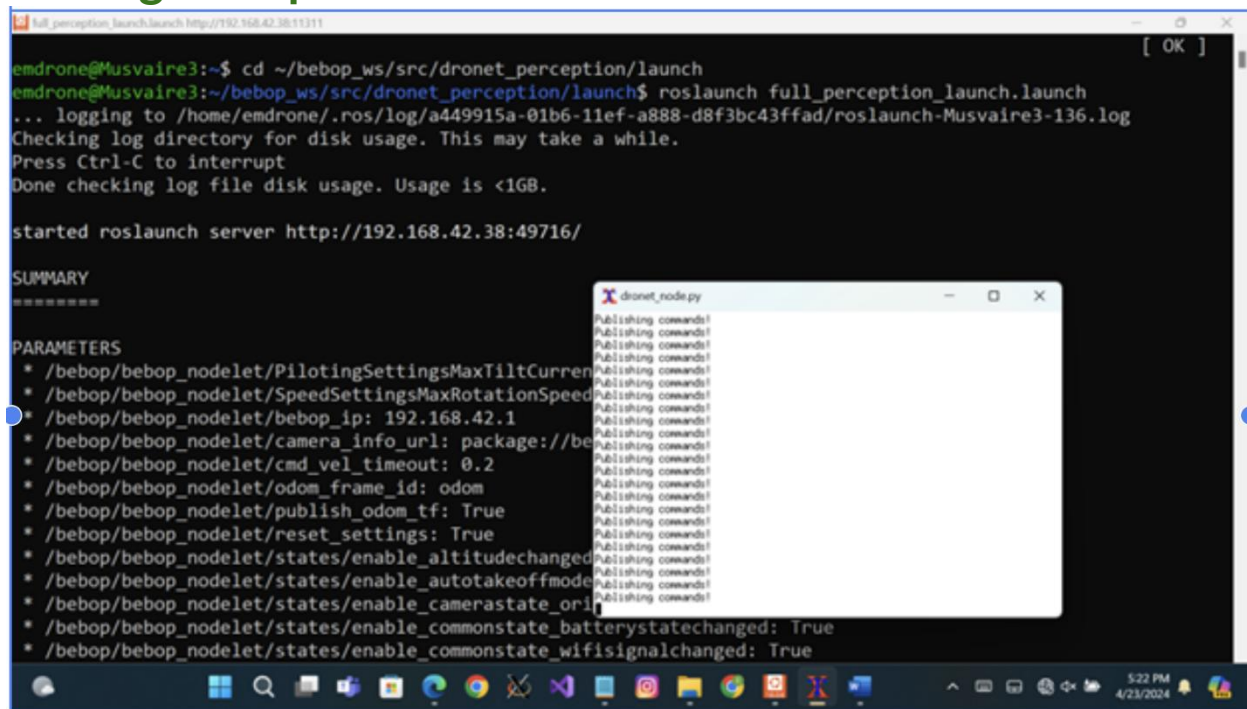
During execution, the packages in the workspace operate as an interconnected pipeline that enables real-time UAV perception and control:

- **Bebop autonomy** publishes the drone’s raw sensor data and receives control commands. It handles camera streams, IMU data, and low-level flight messages, exposing them as ROS topics that other modules can subscribe to.
- **Model perception** subscribes to the camera feed and processes each frame through the trained neural network. It outputs steering-angle predictions or collision-probability estimates as ROS messages, making them available to downstream components.
- **Model control** listens to the perception outputs and converts them into actionable flight commands. It applies control logic—such as smoothing, thresholding, or safety overrides—to generate velocity or attitude commands that are then sent back to the Bebop autonomy node.

- **Graphical user interface** provides a real-time visualization layer. It displays the camera feed, predicted steering angles, collision alerts, and system status, allowing the operator to monitor the UAV's behavior and intervene if necessary.

Together, these modules form a closed-loop system in which perception informs control, control commands drive the drone, and the resulting sensor data feeds back into the perception module. This tight coupling ensures that the UAV can react continuously to its environment while maintaining stable and interpretable behavior.

Testing Bebop Drone 2



```

emdrone@Musvaire3:~$ cd ~/bebop_ws/src/dronet_perception/launch
emdrone@Musvaire3:~/bebop_ws/src/dronet_perception/launch$ roslaunch full_perception_launch.launch
... logging to /home/emdrone/.ros/log/a449915a-01b6-11ef-a888-d8f3bc43ffad/roslaunch-Musvaire3-136.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.42.38:49716/

SUMMARY
*****

PARAMETERS
 * /bebop/bebop_nodelet/PilotingSettingsMaxTiltCurrent
 * /bebop/bebop_nodelet/SpeedSettingsMaxRotationSpeed
 * /bebop/bebop_nodelet/bebop_ip: 192.168.42.1
 * /bebop/bebop_nodelet/camera_info_url: package://be
 * /bebop/bebop_nodelet/cmd_vel_timeout: 0.2
 * /bebop/bebop_nodelet/odom_frame_id: odom
 * /bebop/bebop_nodelet/publish_odom_tf: True
 * /bebop/bebop_nodelet/reset_settings: True
 * /bebop/bebop_nodelet/states/enable_altitudechanged
 * /bebop/bebop_nodelet/states/enable_autotakeoffmode
 * /bebop/bebop_nodelet/states/enable_camerastate_ori
 * /bebop/bebop_nodelet/states/enable_commonstate_batterystatechanged: True
 * /bebop/bebop_nodelet/states/enable_commonstate_wifisignalchanged: True
  
```

Fig. 21. Deployment of the trained perception model onto the Bebop 2 platform using the full-perception launch configuration. The launch file initializes all required ROS nodes, connects to the drone over its default network interface, and activates the perception pipeline responsible for real-time steering and collision-probability inference. During testing, the Bebop 2 streams live camera data to the onboard perception node, which publishes control-ready predictions while the autonomy stack handles communication, state updates, and command execution. This setup demonstrates the complete end-to-end integration of the trained model within the operational UAV system.

Model Output Flow During Deployment

Once the full-perception launch is executed, the Bebop 2 begins streaming live camera frames to the perception node, where each image is processed through the fine-tuned neural network. The model generates either a steering-angle prediction or a collision-probability estimate, which is immediately published as a ROS message. The control module subscribes to this prediction topic and transforms the model's output into actionable flight commands by applying smoothing, safety thresholds, and rate limits. These commands are then forwarded to the Bebop autonomy node, which handles low-level communication with the drone and executes the velocity or

attitude adjustments in real time. This closed-loop flow—from camera input to model inference to control output—enables the UAV to react continuously to its environment while maintaining stable, autonomous behavior.

Deep Navigation Launch

```

emdrone@Musvaire3:~$ cd ~/bebop_ws/src/dronet_control/launch
emdrone@Musvaire3:~/bebop_ws/src/dronet_control/launch$ roslaunch deep_navigation.launch
... logging to /home/emdrone/.ros/log/a31de46a-8f52-11ee-935b-d8f3bc43ffae/roslaunch-Musvaire3-6039.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://192.168.42.38:57716/

SUMMARY
=====

PARAMETERS
* /dronet_control/alpha_velocity: 0.4
* /dronet_control/alpha_yaw: 0.7
* /dronet_control/critical_prob: 1.3
* /dronet_control/max_forward_index: 0.4
* /roscdistro: melodic
* /rosversion: 1.14.13

NODES
/
dronet_control (dronet_control/dronet_control)

ROS_MASTER_URI=http://192.168.42.38:11311

process[dronet_control-1]: started with pid [6191]
[ INFO] [1701330244.795341500]: [/dronet_control]: Initializing Deep Control Node
[ INFO] [1701330244.801091500]: [/dronet_control]: Reading parameters
[ INFO] [1701330246.881707400]: Desired_Forward_Velocity [0-1]: 0.400
[ INFO] [1701330246.882128700]: Desired_Angular_Velocity[0-1]: 0.000

```

Fig. 22A.

```

[ INFO] [1713907558.932086600]: Collision Prob.: 1.000 - OutSteer: 0.346
[ INFO] [1713907558.932273300]: -----
[ INFO] [1713907558.964838700]: Desired_Forward_Velocity [0-1]: 0.000
[ INFO] [1713907558.965099300]: Desired_Angular_Velocity[0-1]: 0.348
[ INFO] [1713907558.965258100]: Collision Prob.: 1.000 - OutSteer: 0.347
[ INFO] [1713907558.965411600]: -----
[ INFO] [1713907558.998087800]: Desired_Forward_Velocity [0-1]: 0.000
[ INFO] [1713907558.998339400]: Desired_Angular_Velocity[0-1]: 0.347
[ INFO] [1713907558.998463100]: Collision Prob.: 1.000 - OutSteer: 0.347
[ INFO] [1713907558.998652500]: -----
[ INFO] [1713907559.031645800]: Desired_Forward_Velocity [0-1]: 0.000
[ INFO] [1713907559.032032600]: Desired_Angular_Velocity[0-1]: 0.347
[ INFO] [1713907559.032201400]: Collision Prob.: 1.000 - OutSteer: 0.347
[ INFO] [1713907559.032593800]: -----
[ INFO] [1713907559.065016700]: Desired_Forward_Velocity [0-1]: 0.000
[ INFO] [1713907559.065243800]: Desired_Angular_Velocity[0-1]: 0.337
[ INFO] [1713907559.065403900]: Collision Prob.: 1.000 - OutSteer: 0.333
[ INFO] [1713907559.065584700]: -----
[ INFO] [1713907559.098115200]: Desired_Forward_Velocity [0-1]: 0.000
[ INFO] [1713907559.098359700]: Desired_Angular_Velocity[0-1]: 0.334
[ INFO] [1713907559.098578600]: Collision Prob.: 1.000 - OutSteer: 0.333
[ INFO] [1713907559.098727300]: -----
[ INFO] [1713907559.131571300]: Desired_Forward_Velocity [0-1]: 0.000
[ INFO] [1713907559.131890100]: Desired_Angular_Velocity[0-1]: 0.334
[ INFO] [1713907559.132169700]: Collision Prob.: 1.000 - OutSteer: 0.333
[ INFO] [1713907559.132435400]: -----

```

Fig. 22B.

Fig. 22A and Fig 22B. Deep Navigation running in real time, showing how the deployed control module interprets each incoming video frame to generate the *desired forward velocity*, *desired angular velocity*, and *collision probability*. When the model detects a high likelihood of collision, the controller immediately reduces the forward velocity to zero, bringing the UAV to a halt. When an object is present but not critically close, the system adjusts the angular velocity to steer left or right and avoid the obstacle while modulating speed. In the absence of obstacles, the UAV maintains a straight trajectory with stable forward motion. This closed-loop behavior demonstrates how perception-driven predictions directly shape safe and adaptive flight decisions during autonomous navigation.

Live Flight Validation of the Control Outputs

To verify that the deployed control module behaved as intended, the system was evaluated through a series of live flight tests in which the Bebop 2 navigated real environments while receiving predictions from the perception model in real time. During these tests, each video frame produced a collision probability, a desired forward velocity, and a desired angular velocity, allowing the UAV to continuously adjust its motion based on the scene ahead. When obstacles appeared close to the drone, the controller consistently reduced the forward velocity to zero, demonstrating correct emergency-halt behavior. When obstacles were present but not immediately threatening, the UAV executed smooth left or right steering adjustments, matching the angular-velocity commands generated by the model. In open, obstacle-free areas, the drone maintained stable forward motion with minimal angular correction, confirming that the perception-driven control loop could support straight-line navigation. These observations validated that the integrated perception and control modules functioned reliably under real-world conditions and that the model's outputs translated into safe, interpretable flight behavior.

Use-Case Modeling

Use-Case Modeling for Perception-Driven UAV Navigation

The UAV's autonomous navigation behavior is governed by the model's continuous assessment of collision risk and steering requirements derived from each incoming camera frame. The system adjusts its forward velocity and angular velocity based on the predicted likelihood of encountering an obstacle. Three representative use cases illustrate how the perception and control modules interact to ensure safe and adaptive flight.

Use Case 1 — High-Risk Collision Scenario

- In this scenario, the camera frame is classified as *class 1 (collision)* with a collision probability of 94.5%. This high-confidence prediction indicates that an object is extremely close to the UAV. The control module responds by reducing the forward velocity to zero, bringing the drone to an immediate halt to prevent impact. This use case demonstrates the system's emergency-stop behavior when the environment presents an imminent threat.

Use Case 2 — No-Collision Scenario with Minor Course Adjustment

- The second scenario is classified as *class 0 (no-collision)* because the frame contains no visual indicators of an impending obstacle. The model outputs a steering angle of -0.123 and a collision probability of 0.00%, prompting the UAV to steer slightly to the left while maintaining a forward velocity of 0.4. This reflects normal navigation behavior, where the drone makes small heading corrections to stay aligned with the path while continuing forward motion.

Use Case 3 — Low-Risk Scenario Requiring Moderate Steering

- In the third scenario, the frame is also classified as *class 0 (no-collision)*, with only a 0.48% chance of becoming a collision frame. The model predicts a steering angle of -0.372 and a forward velocity of 0.381 m/s, causing the UAV to steer more noticeably to the left while still advancing. This behavior represents situations where an object is present in the scene but not close enough to require stopping, allowing the drone to navigate around it smoothly.

Together, these three use cases illustrate the UAV's full decision-making spectrum—from emergency stopping to subtle course corrections—highlighting how perception-driven predictions translate into safe, interpretable, and context-appropriate control actions during autonomous flight.

Model Use Case 1

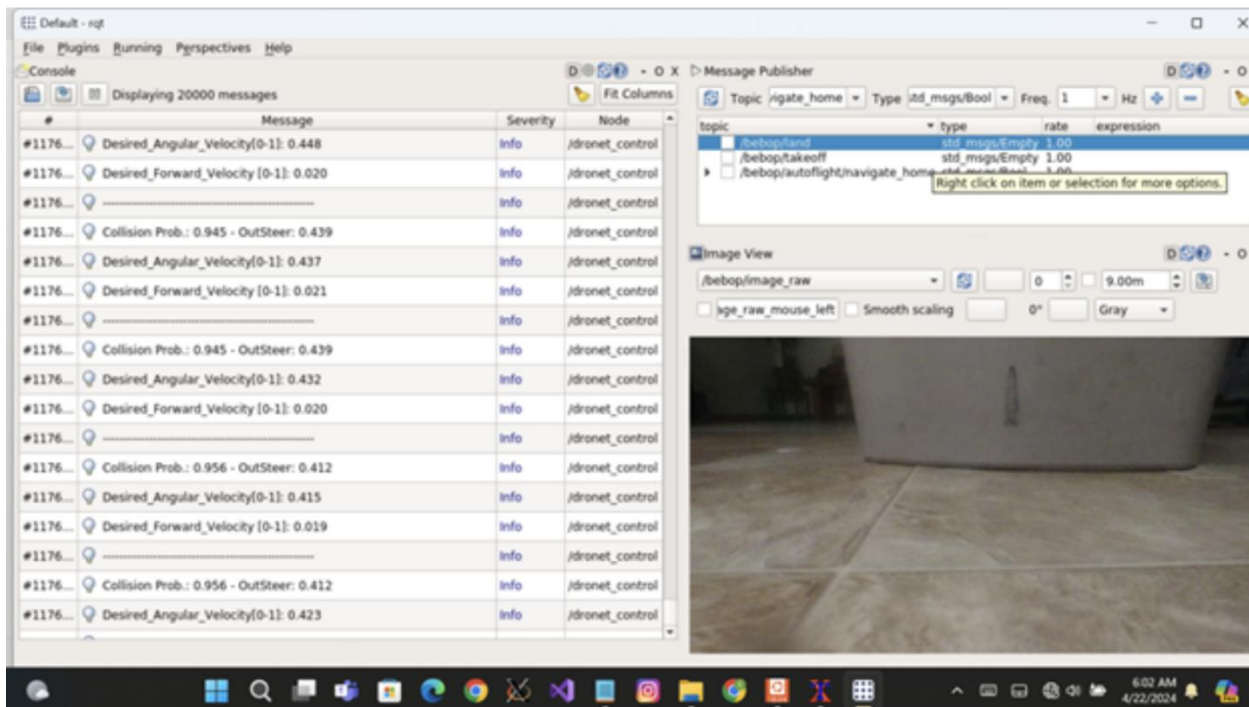


Fig. 23. Use-case modeling of the UAV's obstacle-response behavior. This scenario illustrates how the drone interprets its camera input and adjusts its motion based on the predicted collision risk. In Use Case 1, the incoming image is classified as *class 1 (collision)* with a collision probability of 94.5%, indicating that an object is extremely close to the UAV. This high-risk prediction triggers an immediate safety response in which the drone halts forward motion to prevent impact. The broader use-case model captures the full decision logic: the UAV stops

when an obstacle is very close, slows and steers left or right when an obstacle is within sight, and continues straight ahead when no obstacle is detected.

Model Use Case 2

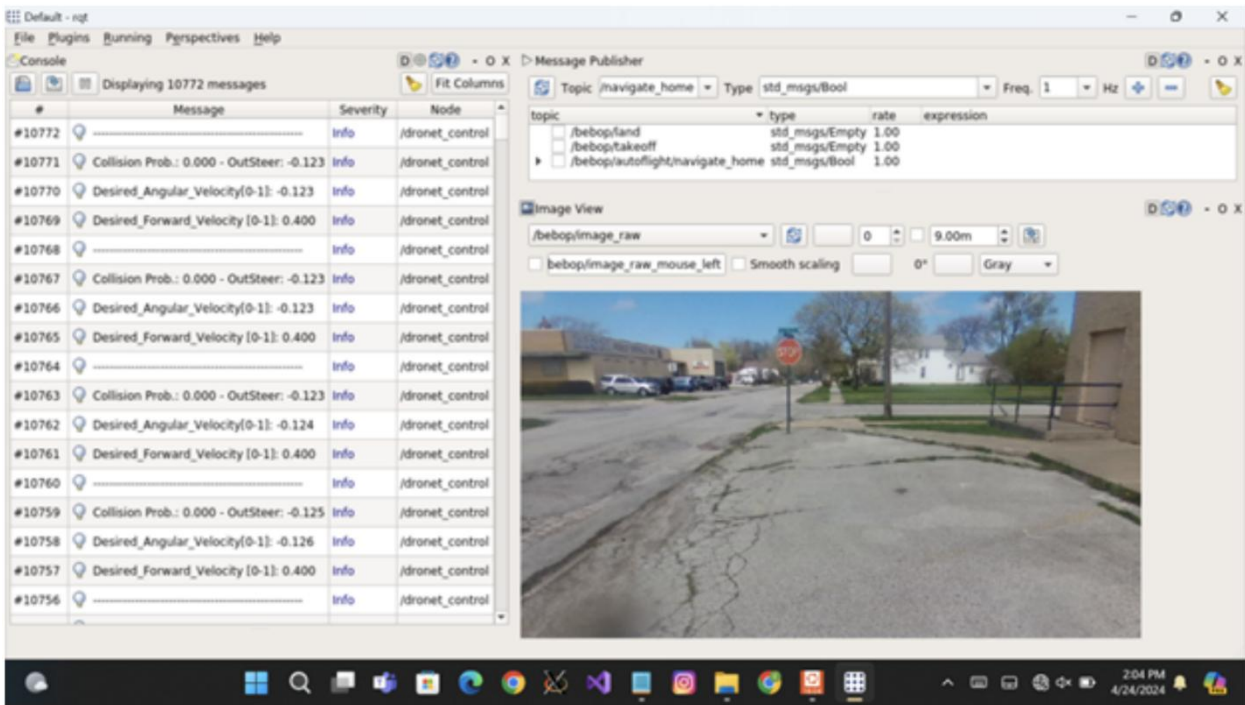


Fig. 24. Use Case 2 illustrates a scenario in which the UAV correctly identifies the scene as *class 0 (no-collision)* because the incoming video frame contains no visual indicators of an imminent obstacle. The model outputs a steering angle of -0.123 , signaling a slight left turn, along with a collision probability of 0.00%. Based on these predictions, the control module commands a forward velocity of 0.4, allowing the drone to continue moving while gently adjusting its heading. This behavior reflects the UAV’s normal navigation mode, where subtle steering corrections maintain course alignment in the absence of collision risk.

Model Use Case 3

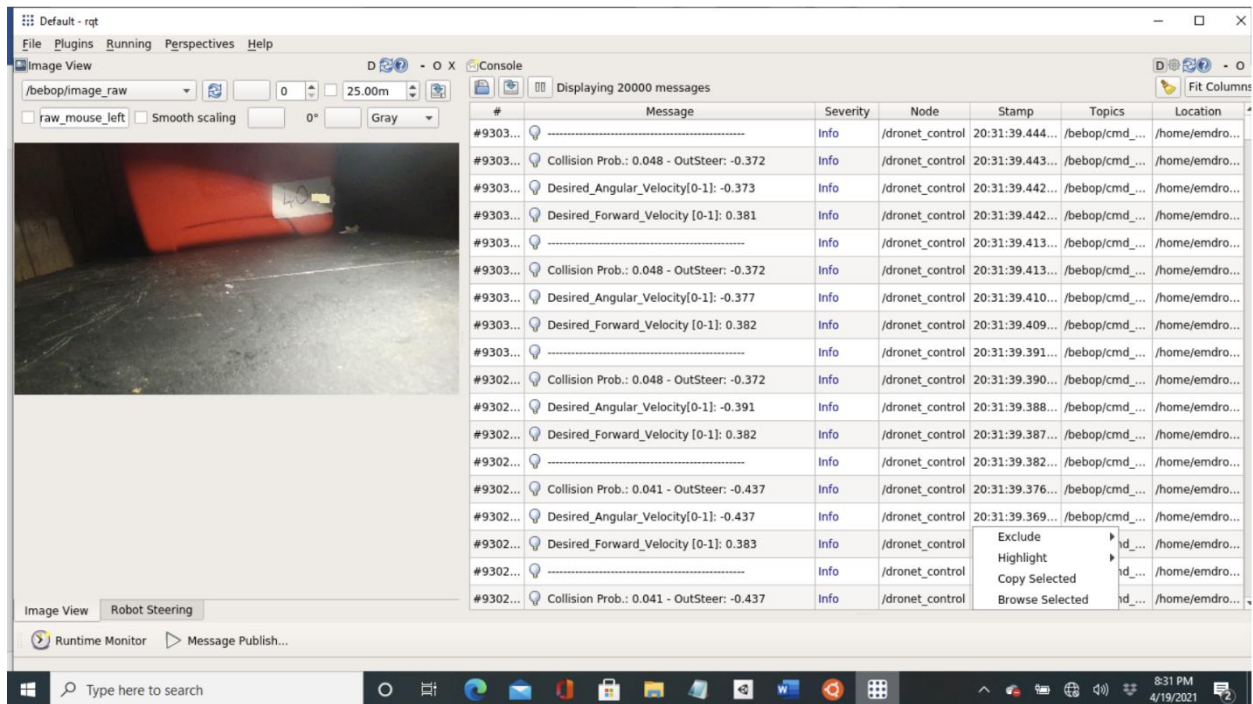


Fig. 25. Use Case 3 demonstrates a no-collision scenario in which the UAV correctly classifies the incoming camera frame as *class 0 (no-collision)*, with only a 0.48% estimated probability of becoming a collision frame. The model outputs a steering angle of -0.372 , indicating a deliberate leftward adjustment, and a forward velocity command of 0.381 m/s. These predictions prompt the control module to guide the drone smoothly to the left while maintaining forward motion, reflecting normal navigation behavior when an object is present in the scene but not close enough to pose an immediate threat.

Controlling the Drone

- The outputs of model are used to command the UAV to move on a plane with forward velocity v_k and steering angle θ_k .

$$v_k = (1 - \alpha)v_{k-1} + \alpha(1 - p_t)V_{max}$$

- When probability of collision $p_t = 0$, forward velocity = V_{max}
- When probability of collision $p_t = 1$, forward velocity = 0
- Steering angle is calculated by mapping the predicted scaled steering s_k into a rotation around the z-axis.

$$\theta_k = (1 - \beta)\theta_{k-1} + \beta \frac{\pi}{2} s_k$$

- α and β perform low pass filtering on the velocity and steering angle to provide smooth, continuous inputs. The paper uses $\alpha = 0.7$ and $\beta = 0.5$.

conclusion

Model	EVA	RMSE	Avg. accuracy	F-1 score	Num. Layers
Drone Model	0.865	0.181	95.3%	0.895	8

Figure 23. Summary of the drone model's performance metrics, showing its effectiveness across both regression and classification tasks. The evaluation reports an explained variance (EVA) of 0.865 and a root-mean-square error (RMSE) of 0.181 for steering-angle prediction, indicating strong regression accuracy. For the collision-classification task, the model achieves an average accuracy of 95.3% and an F-1 score of 0.895, reflecting balanced precision and recall in detecting collision versus no-collision frames. The architecture consists of eight layers, demonstrating that a relatively compact network can deliver high-quality perception performance suitable for real-time UAV navigation.